

Pet Territory Wars — Project Conventions v1.1

Backend: Go

Mobile: Flutter

Database: PostgreSQL + PostGIS

Architecture: Modular Monolith + Stateless Game Engine

Scope: MVP

Status: Active

1. Amaç

Bu doküman Pet Territory Wars projesinin ortak geliştirme standartlarını tanımlar.

Temel hedefler:

- Kod tabanını okunabilir ve sürdürülebilir tutmak
- Business kurallarını teknik katmanlardan ayırmak
- Gereksiz soyutlama ve overengineering'i önlemek
- Test edilebilir ve değiştirilebilir bir yapı kurmak
- Aynı anda binlerce kullanıcının işlem yaptığı durumda güvenilirliği korumak
- Backend, worker, API ve mobil ekipleri arasında ortak dil oluşturmak

Bu dokümanda tanımlanan kurallar MVP boyunca varsayılan proje standardıdır.

2. Mimari Özet

Sistem aşağıdaki ana akışa sahiptir:

```
Flutter Mobile
  ↓
REST API
  ↓
Application Use Case
  ↓
Game Engine
  ↓
Repository
  ↓
PostgreSQL
```

Asenkron işlemler:

PostgreSQL Job Queue
↓
Walk Worker
↓
Application Use Case
↓
Game Engine
↓
PostgreSQL Transaction
↓
Outbox Worker

Temel mimari yaklaşım:

Tek backend uygulaması
Tek game engine
Birden fazla küçük bileşen
Stateless worker'lar
Versiyonlu RuleSet
PostgreSQL source of truth

3. Katman Sorumlulukları

3.1 Mobile

Flutter uygulamasının sorumlulukları:

- UI göstermek
- GPS verisi toplamak
- Walk lifecycle ekranlarını yönetmek
- API ile iletişim kurmak
- Yerel geçici veri saklamak
- Backend sonucunu haritada göstermek

Flutter authoritative business rule çalıştırmaz.

Flutter içinde hesaplanmaması gerekenler:

Walk geçerli mi?
Hex ele geçirildi mi?
Dominance ne kadar değişti?
Hex sahibi kim oldu?
Skor kaç arttı?
Decay ne kadar uygulandı?

Mobil uygulama yalnızca geçici önizleme gösterebilir. Nihai karar backend'e aittir.

3.2 API Layer

API katmanının sorumlulukları:

- HTTP request almak
- Authentication bilgilerini çözmek
- Request DTO doğrulamak
- Application use case çağırarak
- Domain sonucunu response DTO'ya çevirmek
- HTTP durum kodunu belirlemek

API katmanı business rule uygulamaz.

Yanlış:

```
if req.DistanceMeters < 300 {  
    return ErrInvalidWalk  
}
```

Doğru:

```
result, err := finishWalkUseCase.Execute(ctx, input)
```

Minimum mesafe Calculator Rules ve Game Engine sorumluluğudur.

3.3 Application Layer

Application katmanı kullanım senaryolarını yönetir.

Sorumlulukları:

- Repository'lerden veri toplamak
- Aktif RuleSet'i yüklemek
- Engine input'unu hazırlamak
- Game Engine'i çağırarak
- Transaction sınırını yönetmek
- Engine sonucunu kalıcı hale getirmek
- Outbox kayıtlarını oluşturmak
- Retry ve conflict akışını yönetmek

Application katmanı yeni oyun matematiği üretmez.

Örnek use case'ler:

```
StartWalk
UploadWalkPoints
FinishWalk
ResolveWalk
GetWalkResult
GetTerritoryViewport
GetLeaderboard
```

3.4 Game Engine

Game Engine iş kurallarının merkezidir.

Girdi:

```
Walk input
Current territory state
Current score state
RuleSet
Evaluation time
```

Çıktı:

```
Validation result
Hex changes
Score changes
Domain events
Calculation metadata
```

Engine:

- Database'e erişmez
- Repository kullanmaz
- HTTP çağrısı yapmaz
- Redis kullanmaz
- Queue bilmez
- Log göndermez
- Push notification göndermez
- `time.Now()` çağırır
- Random sayı üretmez

Temel kural:

```
Same Input + Same State + Same RuleSet = Same Output
```

3.5 Repository Layer

Repository yalnızca veri erişimi yapar.

Sorumlulukları:

- Veri okumak
- Veri yazmak
- Optimistic locking uygulamak
- Query sonucunu persistence modeline çevirmek
- Database hatalarını anlamlı teknik hatalara dönüştürmek

Repository şunları yapmaz:

```
Dominance hesaplamak
Walk validasyonu yapmak
Skor kararı vermek
Domain event üretmek
Notification göndermek
```

3.6 Infrastructure Layer

Teknik implementasyonlar burada bulunur.

Örnek:

```
PostgreSQL repository implementasyonları
H3 adapter
HTTP server
Job worker
Outbox worker
Redis cache
Logger
Metrics
Tracing
Configuration loader
```

Infrastructure domain kararlarını vermez.

4. Önerilen Proje Yapısı

```
cmd/
├─ api/
└─ main.go
```

```
|— walk-worker/
|   └─ main.go
|— outbox-worker/
|   └─ main.go
|— simulator/
|   └─ main.go

internal/
|— api/
|   └─ dto/
|   └─ handler/
|   └─ mapper/
|   └─ middleware/
|   └─ response/
|
|— application/
|   └─ walk/
|   └─ territory/
|   └─ leaderboard/
|   └─ player/
|   └─ transaction/
|
|— domain/
|   └─ player/
|   └─ pet/
|   └─ walk/
|   └─ territory/
|   └─ season/
|   └─ rules/
|   └─ event/
|
|— gameengine/
|   └─ engine.go
|   └─ input.go
|   └─ result.go
|   └─ rules.go
|   └─ validation.go
|   └─ movement.go
|   └─ territory.go
|   └─ scoring.go
|   └─ events.go
|   └─ types.go
|
|— repository/
|   └─ player.go
|   └─ walk.go
|   └─ territory.go
|   └─ score.go
|   └─ rules.go
|   └─ jobs.go
```

```
|
|
|— infrastructure/
|   |— postgres/
|   |— h3adapter/
|   |— worker/
|   |— outbox/
|   |— cache/
|   |— logging/
|   |— metrics/
|   |— tracing/
|   └─ config/
|
|— shared/
|   |— clock/
|   |— id/
|   |— pagination/
|   └─ validation/
|
migrations/
configs/
testdata/
docs/
└─ adr/
```

4.1 Shared Paketi Kuralı

`shared` paketi genel çöp kutusu olmayacaktır.

Aşağıdaki isimler yasaktır:

```
utils
helpers
common
misc
```

Bir kodun sorumluluğu varsa o sorumluluğa göre isimlendirilmelidir.

Yanlış:

```
shared/utils
```

Doğru:

```
shared/id
shared/clock
shared/pagination
```

5. Bağımlılık Yönleri

İzin verilen temel bağımlılık yönü:

```
API
↓
Application
↓
Domain / Game Engine
```

Application ayrıca kendi tanımladığı repository sözleşmelerini kullanabilir.

Infrastructure bu sözleşmeleri uygular.

```
Application → Repository Interface
Infrastructure → Repository Implementation
```

Yasak bağımlılıklar:

```
Domain → Infrastructure
Game Engine → PostgreSQL
Game Engine → HTTP
Game Engine → Redis
Repository → API DTO
Domain → Flutter modeli
```

6. Interface Kuralları

Go'da interface yalnızca gerçek ihtiyaç varsa tanımlanır.

Temel kural:

Interface implement eden tarafta değil, kullanan tarafta tanımlanır.

Örnek:


```
type TerritoryRepository interface {
    GetByHexIDs(
        ctx context.Context,
        cityID uuid.UUID,
        hexIDs []int64,
    ) ([]territory.HexState, error)
}
```

Interface'ler küçük tutulmalıdır.

Yanlış:

```
type Repository interface {
    GetPlayer(...)
    SavePlayer(...)
    GetWalk(...)
    SaveWalk(...)
    GetHexes(...)
    SaveScores(...)
    PublishEvent(...)
}
```

Doğru:

```
WalkRepository
TerritoryRepository
ScoreRepository
RuleSetRepository
```

Her struct için interface oluşturulmaz.

Aşağıdaki durumda interface gereklidir:

- Alternatif implementasyon varsa
- Test double gerekiyorsa
- Harici bağımlılığı izole ediyorsa
- Application ile infrastructure sınırını belirliyorsa

7. Dependency Injection

Bağımlılıklar constructor üzerinden verilir.

```
type ResolveWalkUseCase struct {
    walks      WalkRepository
```

```
territories TerritoryRepository
scores      ScoreRepository
engine      GameEngine
}
```

```
func NewResolveWalkUseCase(
    walks WalkRepository,
    territories TerritoryRepository,
    scores ScoreRepository,
    engine GameEngine,
) *ResolveWalkUseCase {
    return &ResolveWalkUseCase{
        walks:      walks,
        territories: territories,
        scores:      scores,
        engine:      engine,
    }
}
```

Global mutable dependency kullanılmaz.

Service locator kullanılmaz.

İlk MVP'de dependency injection framework kullanılmaz.

Bağlantılar `main.go` veya ayrı bootstrap paketi içinde açıkça kurulur.

8. Domain Model Kuralları

Domain modelleri framework bağımsızdır.

Domain struct'larında aşağıdakiler bulunmaz:

```
json tag
db tag
gorm tag
validation tag
HTTP status
SQL sorgusu
```

Yanlış:

```
type Player struct {
    ID uuid.UUID `json:"id" db:"id"`
}
```

Doğru:

```
type Player struct {
    ID PlayerID
}
```

Domain tipleri mümkün olduğunda özel tiplerle ifade edilir:

```
type PlayerID uuid.UUID
type WalkID uuid.UUID
type HexID uint64
type Dominance int
```

Bu yaklaşım yanlış parametre kullanımını azaltır.

9. DTO ve Mapper Kuralları

API DTO, domain modeli ve database modeli birbirinden ayrıdır.

Akış:

```
HTTP Request DTO
  ↓
API Mapper
  ↓
Application Input
  ↓
Domain / Engine
  ↓
Application Result
  ↓
API Response Mapper
  ↓
HTTP Response DTO
```

Domain modeli doğrudan JSON'a serialize edilmez.

Database modeli doğrudan API response olarak dönmez.

Mapper'lar business rule uygulamaz.

Mapper yalnızca şekil dönüşümü yapar.

10. Repository Kuralları

Repository fonksiyon isimleri domain amacını ifade etmelidir.

Örnek:

```
GetByID  
GetByHexIDs  
Create  
UpdateWithVersion  
ApplyScoreDeltas  
ClaimPendingJobs
```

Aşağıdaki tarzdan kaçınılır:

```
DoStuff  
ProcessData  
HandleWalk  
RunQuery
```

Repository transaction başlatmaz.

Transaction application katmanı tarafından sağlanır.

Repository aynı transaction handle'ını kullanabilmelidir.

11. Transaction Standartları

Transaction yalnızca application katmanında açılır.

Temel işlem:

1. Transaction dışında state oku
2. Engine hesabını transaction dışında çalıştır
3. Kısa transaction aç
4. ExpectedVersion ile update uygula
5. Score delta uygula
6. Audit kayıtlarını yaz
7. Outbox event yaz

8. Walk state deęiřtir
9. Commit

Transaction içinde yapılmayacaklar:

HTTP çağrısı
Push gönderme
Object storage upload
Uzun engine hesaplaması
Sleep
Harici servis retry

Birden fazla hex güncellenirken deterministik sıra kullanılır:

hex_id ASC

12. Error Yönetimi

Hatalar üç gruba ayrılır.

12.1 Business Result

Normal domain sonucudur.

Örnek:

WALK_TOO_SHORT
INVALID_GPS_ROUTE
OUTSIDE_ACTIVE_CITY
WALK_ALREADY_RESOLVED

Bunlar her zaman Go `error` olmak zorunda değildir.

Engine validation result içinde dönebilir.

12.2 Application Error

Use case'in tamamlanamadığını gösterir.

Örnek:

```
WalkNotFound  
PlayerSuspended  
RuleSetUnavailable  
ConcurrencyConflict
```

Typed error kullanılmalıdır.

12.3 Technical Error

Infrastructure kaynaklıdır.

Örnek:

```
DatabaseUnavailable  
QueryTimeout  
H3ConversionFailed  
SerializationFailed
```

Hatalar wrap edilmelidir:

```
return fmt.Errorf("load territory states: %w", err)
```

Hata mesajına hassas veri eklenmez.

Genel `errors.New("error")` kullanılmaz.

13. Go Coding Standards

13.1 Go Sürümü

Proje başlangıcında seçilen stabil Go sürümü `go.mod` içinde sabitlenir.

Go sürümü upgrade'i ayrı pull request ile yapılır.

13.2 Formatting

Bütün Go kodu:

```
gofmt  
goimports
```

ile formatlanmalıdır.

Format tartışması code review konusu değildir.

13.3 Lint

Minimum araçlar:

```
go vet
staticcheck
golangci-lint
```

CI pipeline lint hatasında başarısız olur.

13.4 Package İsimleri

Package isimleri:

- Küçük harf olmalı
- Tek kelime tercih edilmeli
- Çoğul olmamalı
- Alt çizgi içermemeli

Doğru:

```
walk
territory
season
```

Yanlış:

```
walks
territory_service
gameEngine
```

13.5 Dosya İsimleri

Dosya isimleri küçük harf ve snake_case olabilir:

```
resolve_walk.go  
territory_repository.go  
walk_handler.go
```

Dosya adı içeriğin sorumluluğunu ifade etmelidir.

13.6 Struct İsimleri

İsimler açık olmalıdır:

```
ResolveWalkUseCase  
PostgresWalkRepository  
WalkProcessingJob
```

Gereksiz son ek kullanılmaz:

```
Manager  
Helper  
Util  
Processor  
Handler
```

`Handler` yalnızca HTTP veya mesaj handler'ı gibi gerçek handler yapılarında kullanılmalıdır.

13.7 Receiver Kuralları

Receiver ismi kısa ve tipe uygun olmalıdır:

```
func (w Walk) IsResolved() bool  
func (e Engine) ResolveWalk(...)  
func (r Repository) GetByID(...)
```

`this` veya `self` kullanılmaz.

Pointer receiver şu durumlarda kullanılır:

- Struct değiştiriliyorsa
- Struct büyükse
- Kopyalanması istenmiyorsa
- Metot seti tutarlılığı gerekiyorsa

Value receiver şu durumlarda kullanılabilir:

- Küçük immutable value object
- Değişiklik yapılmayan tip
- Kopyalama maliyeti düşük tip

Aynı tipte pointer ve value receiver karışık kullanılmamalıdır.

13.8 Context Kullanımı

I/O yapan public application ve repository fonksiyonlarının ilk parametresi:

```
ctx context.Context
```

olmalıdır.

```
func (r *Repository) GetByID(  
    ctx context.Context,  
    id WalkID,  
    ) (Walk, error)
```

Context:

- Struct içinde saklanmaz
- `nil` gönderilmez
- Business data taşımak için kullanılmaz
- Timeout ve cancellation için kullanılır

Engine saf hesaplama yaptığı için context gerekmebilir.

13.9 Goroutine Kullanımı

Goroutine yalnızca lifecycle ve hata yönetimi açıkça kontrol edilebiliyorsa açılır.

Yasak:

```
go doSomething()
```

ve sonucunu takip etmemek.

Kurallar:

- Goroutine leak olmamalı
- Context cancellation desteklenmeli

- Panic recovery worker sınırında yapılmalı
- Sınırsız concurrency kullanılmamalı
- Worker pool veya semaphore tercih edilmeli

13.10 Slice ve Map Kuralları

Map iteration sırası deterministik değildir.

Engine sonucu map sırasına bağlı olmamalıdır.

Hex'ler işlenmeden önce sıralanır:

```
sort.Slice(hexIDs, func(i, j int) bool {  
    return hexIDs[i] < hexIDs[j]  
})
```

Büyük slice kapasitesi biliniyorsa önceden ayrılır:

```
changes := make([]HexChange, 0, len(visits))
```

13.11 Panic Kullanımı

Normal hata akışında panic kullanılmaz.

Panic yalnızca:

- Program başlatma sırasında zorunlu dependency yoksa
- Kesinlikle ulaşılamaması gereken invariant bozulduysa

düşünülebilir.

Worker ve HTTP server sınırında panic recovery bulunmalıdır.

13.12 Magic Number

Business değerleri kod içine yazılmaz.

Yanlış:

```
damage := 15
```

Doğru:

```
damage := rules.Territory.EnemyAttackDamage
```

Teknik sabitler anlamlı isimle tanımlanabilir:

```
const sha256HexLength = 64
```

14. Naming Convention

Boolean

```
IsActive  
HasOwner  
CanCapture  
ShouldRetry
```

Zaman

```
CreatedAt  
UpdatedAt  
ResolvedAt  
EvaluatedAt
```

Kimlik

```
PlayerID  
WalkID  
CityID  
HexID
```

Sayaç

```
CaptureCount  
AttemptCount  
ActiveHexCount
```

Mesafe

Birim isimde açıkça belirtilir:

```
DistanceMeters
SpeedMPS
AccuracyMeters
DurationSeconds
```

`Distance`, `Speed`, `Duration` tek başına kullanılmaz.

15. Configuration ve Secret Yönetimi

Config ve secret birbirinden ayrılır.

Config örnekleri

```
HTTP port
Database pool size
Worker batch size
Log level
Job retry count
```

Secret örnekleri

```
Database password
JWT signing key
OAuth client secret
API keys
```

Secret'lar:

- Repository'ye commit edilmez
- Loglanmaz
- Config dosyasında açık tutulmaz
- Environment veya secret manager'dan alınır
- Düzenli rotate edilebilir olmalıdır

`.env` yalnızca local development için kullanılabilir.

Production secret'ları `.env` dosyasıyla yönetilmez.

16. Security Standards

16.1 Input Validation

API katmanı teknik doğrulama yapar:

```
Required field  
String length  
Valid UUID  
Payload size  
Array limit  
Timestamp format
```

Engine business validation yapar:

```
Walk distance  
Walk duration  
GPS speed  
GPS accuracy  
City boundary
```

16.2 SQL Güvenliği

Bütün sorgular parametrelili olmalıdır.

Yanlış:

```
query := "SELECT * FROM walks WHERE id = '" + id + "'"
```

Doğru:

```
row := db.QueryRowContext(  
    ctx,  
    "SELECT ... WHERE id = $1",  
    id,  
)
```

Dynamic SQL yalnızca whitelist ile oluşturulur.

16.3 Hassas Veri

Aşağıdakiler loglanmaz:

```
Raw GPS noktaları  
Access token  
Refresh token  
Authorization header  
Password  
OAuth secret  
Tam cihaz integrity payload
```

Player ID ve Walk ID loglanabilir; kişisel veriler maskelenmelidir.

16.4 Authorization

Authentication başarılı olması tek başına erişim izni anlamına gelmez.

Her endpoint ownership kontrolü yapılmalıdır.

Örnek:

```
Oyuncu yalnızca kendi Walk detayını okuyabilir.
```

Admin yetkileri normal kullanıcı endpoint'lerinden ayrılır.

16.5 Rate Limiting

Rate limit API Design aşamasında endpoint bazlı tanımlanacaktır.

Temel kategoriler:

```
Authentication  
Walk point upload  
Walk finish  
Map viewport  
Leaderboard  
Profile
```

16.6 Dependency Security

Bağımlılıklar minimum tutulur.

Kurallar:

- Kullanılmayan dependency kaldırılır
- Güvenlik taraması CI'da çalışır
- Kritik açıklar hızlı güncellenir
- Versiyonlar kontrolsüz şekilde latest kullanılmaz
- Fork veya terk edilmiş kütüphane dikkatle değerlendirilir

Araç örnekleri:

```
govulncheck  
Dependabot veya Renovate
```

17. Logging Standards

Loglar structured JSON formatında olmalıdır.

Her request veya job için mümkün olduğunda:

```
trace_id  
request_id  
player_id  
walk_id  
job_id  
rule_set_version  
engine_version
```

alanları eklenir.

Örnek:

```
logger.Info(  
    "walk resolved",  
    "walk_id", walkID,  
    "player_id", playerID,  
    "captured_hex_count", captured,  
)
```

Log seviyeleri:

```
DEBUG → geliştirme ayrıntıları  
INFO → normal önemli işlemler
```

WARN → beklenmeyen fakat işlem devam eden durum
ERROR → işlem başarısızlığı

Aynı hata birden fazla katmanda tekrar tekrar loglanmaz.

Genellikle hata sistem sınırında loglanır:

- HTTP handler
- Worker loop
- Scheduled job runner

18. Observability Standards

Observability üç temel parçadan oluşur:

Logs
Metrics
Traces

18.1 Metrics

Minimum backend metrikleri:

```
http_requests_total  
http_request_duration_seconds  
walks_submitted_total  
walks_resolved_total  
walks_rejected_total  
walk_processing_duration_seconds  
pending_walk_jobs  
oldest_pending_job_age_seconds  
territory_conflict_total  
engine_calculation_duration_seconds  
outbox_pending_total  
database_query_duration_seconds
```

Yüksek cardinality değerleri metric label olarak kullanılmaz.

Yanlış:

```
player_id label  
walk_id label  
hex_id label
```


Bunlar log ve trace alanıdır.

18.2 Tracing

OpenTelemetry uyumlu tracing tercih edilir.

Trace sınırları:

```
HTTP request
Application use case
Database query
Job processing
Outbox processing
```

Game Engine için tek span yeterlidir:

```
gameengine.resolve_walk
```

Her hex için ayrı span oluşturulmaz.

Bu trace hacmini patlatır.

18.3 Health Checks

Endpoint'ler:

```
/health/live
/health/ready
```

Liveness yalnızca process'in çalıştığını gösterir.

Readiness:

- Database erişimi
- Zorunlu dependency durumu
- Worker'ın iş alabilme durumu

gibi kontrolleri içerebilir.

19. Performance Standards

19.1 Önce Ölçüm

Performans optimizasyonu varsayımla yapılmaz.

Araçlar:

```
pprof
benchmarks
EXPLAIN ANALYZE
database metrics
load tests
```

19.2 Database

Kurallar:

- `SELECT *` kullanılmaz
- Yalnızca gerekli kolonlar okunur
- N+1 sorgular engellenir
- Batch sorgular tercih edilir
- Uzun transaction kullanılmaz
- Index gerçek query pattern'e göre eklenir
- Pagination kullanılmadan büyük listeler dönülmez

19.3 Memory

Raw GPS verisi çok büyükse bütün rota gereksiz kopyalanmamalıdır.

Mümkünse:

- Slice kapasitesi önceden ayrılır
- Tekrarlı serialization azaltılır
- Büyük payload limitlenir
- Batch processing kullanılır

Ancak erken aşamada okunabilirlik uğruna küçük allocation'lar kabul edilebilir.

19.4 Worker Concurrency

Worker concurrency config ile kontrol edilir.

Sınırsız goroutine açılmaz.

İzlenecek metrikler:

```
Queue length
Processing duration
Database pool saturation
Conflict rate
Retry rate
```

Worker sayısı yalnızca CPU'ya göre artırılmaz.

Database kapasitesi ve hot hex conflict oranı da dikkate alınır.

19.5 API Payload

Walk points batch halinde gönderilir.

Kurallar:

- Maksimum batch point sayısı
- Maksimum request payload
- Compression desteği
- Idempotent batch sequence

API Design aşamasında kesinleştirilir.

Harita endpoint'i bütün şehir verisini göndermez.

Viewport veya H3 cell listesi bazlı çalışır.

20. Test Standartları

20.1 Test Piramidi

Öncelik:

```
Game Engine unit tests
Application use case tests
Repository integration tests
API integration tests
End-to-end smoke tests
```

20.2 Table-Driven Tests

Go testleri mümkün olduğunda table-driven yazılır.

```
func TestApplyDecay(t *testing.T) {
    tests := []struct {
        name      string
        current   int
        days      int
        expected  int
    }{
        {
            name:      "applies one day decay",
            current:   100,
            days:      1,
            expected:  95,
        },
        // ...
    }
}
```

20.3 Test Naming

Test<Function>_<Scenario>

Örnek:

```
TestResolveHex_CapturesEmptyHex
TestApplyDecay_StopsAtMinimumDominance
TestFinishWalk_ReturnsExistingResultForDuplicateRequest
```

20.4 Golden Tests

Tam Walk senaryoları için golden test kullanılabilir.

```
testdata/
├─ empty_capture_input.json
├─ empty_capture_expected.json
├─ enemy_attack_input.json
├─ enemy_attack_expected.json
```

RuleSet değişikliğinde fark açıkça incelenir.

20.5 Integration Tests

Repository testleri gerçek PostgreSQL üzerinde çalışmalıdır.

SQLite, PostgreSQL davranışını birebir temsil etmediği için ana integration test database'i olarak kullanılmaz.

Test database için container yaklaşımı kullanılabilir.

20.6 Race Test

CI veya düzenli pipeline'da:

```
go test -race ./...
```

çalıştırılmalıdır.

Özellikle worker ve cache kodları race açısından test edilir.

20.7 Benchmark

Sadece kritik alanlarda benchmark yazılır:

```
Route-to-H3 conversion  
ResolveWalk  
Large hex lookup mapping  
RuleSet parsing
```

Benchmark sonuçları ölçüm amacıyla kullanılır; her fonksiyon için benchmark yazılmaz.

21. Git Workflow

21.1 Branch Yaklaşımı

MVP için sade trunk-based yaklaşım önerilir.

Ana branch:

```
main
```

Kısa ömürlü branch'ler:

```
feature/walk-finish  
fix/territory-conflict  
refactor/rules-loader
```

Uzun ömürlü development branch tutulmaz.

21.2 Pull Request

Her değişiklik pull request üzerinden merge edilir.

PR:

- Tek bir ana amacı taşımali
- Gereksiz unrelated değişiklik içermemeli
- Küçük ve incelenabilir olmalı
- Migration içeriyorsa açıkça belirtilmeli
- Breaking change varsa belirtilmeli
- Test sonucu içermeli

21.3 Commit Mesajları

Conventional Commits formatı kullanılabilir:

```
feat: add asynchronous walk resolution  
fix: prevent duplicate score application  
refactor: isolate h3 adapter  
test: add ownership transfer scenarios  
docs: update database conventions  
chore: upgrade postgres driver
```

Commit mesajı yapılan değişikliği açıkça ifade etmelidir.

21.4 Merge Stratejisi

Tercih:

```
Squash merge
```

Böylece main branch geçmişisi sade kalır.

Büyük migration veya anlamlı birden fazla commit korunacaksa rebase merge düşünülebilir.

21.5 Protected Main

`main` branch:

- Doğrudan push kabul etmez
- CI geçmeden merge olmaz
- En az bir review gerektirir
- Migration değişikliklerinde dikkatli review gerektirir

Tek geliştirici olursa bile PR yaklaşımı değişiklik geçmişini düzenli tutar.

22. CI Standartları

Her pull request'te minimum:

```
go test ./...
go test -race ./...
go vet ./...
staticcheck ./...
golangci-lint run
govulncheck ./...
migration validation
build
```

çalışmalıdır.

Flutter tarafında:

```
flutter analyze
flutter test
dart format check
```

çalıştırılmalıdır.

23. Migration Standartları

Database migration'ları:

- Version control altında tutulur

- Manuel production değişikliği yapılmaz
- Uygulamadan bağımsız çalışabilir
- Geriye uyumluluk dikkate alınır
- Büyük tablo değişiklikleri iki aşamalı yapılır

Migration isim örneği:

```
000001_create_players.up.sql
000002_create_walks.up.sql
000003_create_territory_hexes.up.sql
```

Destructive değişiklik:

1. Yeni kolon ekle
2. Kod iki yapıyla uyumlu hale gelir
3. Veri migrate edilir
4. Eski kullanım kaldırılır
5. Eski kolon sonraki deploy'da silinir

24. API Convention Temelleri

Detaylar API Design v1.0'da belirlenecek ancak temel kurallar:

```
REST
/api/v1
JSON
ISO 8601 UTC timestamps
UUID public IDs
H3 ID JSON içinde string
Consistent error envelope
Idempotency key
Cursor pagination
```

Domain modeli doğrudan response olarak dönmez.

25. Flutter Convention Temelleri

Detaylar Mobile Architecture dokümanında belirlenecek.

Temel kurallar:

- Business rule mobilde çoğaltılmaz
- API DTO ile UI modeli ayrılabilir

- GPS toplama ayrı servis içinde olur
- State yönetimi tek yaklaşımda standartlaşır
- Repository pattern yalnızca gerçek fayda sağlıyorsa kullanılır
- Network retry idempotency ile uyumlu olur
- Token ve hassas veri secure storage'da tutulur
- Raw GPS loglanmaz

26. Code Review Kontrol Listesi

Her PR için kontrol:

Business rule doğru katmanda mı?
Yeni magic number var mı?
Domain modeli framework etiketi taşıyor mu?
Transaction gereğinden uzun mu?
Repository business karar veriyor mu?
Yeni sorgunun index ihtiyacı var mı?
Idempotency korunuyor mu?
Concurrency conflict düşünülmüş mü?
Hassas veri loglanıyor mu?
Testler edge case içeriyor mu?
Yeni abstraction gerçekten gerekli mi?

27. Definition of Done

Bir iş tamamlanmış sayılabilmesi için:

1. Kod doğru katmanda bulunmalı.
2. Business rule Engine veya ilgili domain alanında olmalı.
3. DTO, domain ve persistence modelleri ayrılmış olmalı.
4. Transaction Application katmanında yönetilmeli.
5. Idempotency değerlendirilmiş olmalı.
6. Concurrent update senaryosu düşünülmüş olmalı.
7. Unit veya integration testleri yazılmış olmalı.
8. Lint ve build başarılı olmalı.
9. Structured log ve gerekli metrikler eklenmiş olmalı.
10. Hassas veriler korunmuş olmalı.
11. Migration gerekiyorsa repository'ye eklenmiş olmalı.
12. Dokümantasyon veya ADR gerekiyorsa güncellenmiş olmalı.
13. Gereksiz interface veya abstraction eklenmemiş olmalı.
14. Error handling açık ve typed olmalı.
15. Performans açısından sınırsız sorgu, payload veya goroutine bulunmamalı.

28. Yasaklar

Projede aşağıdakiler kullanılmayacaktır:

- Global mutable state
- Business rule içinde SQL
- Repository içinde oyun matematiği
- API handler içinde dominance hesabı
- Flutter içinde authoritative capture hesabı
- Hardcoded RuleSet değerleri
- Kontrolsüz goroutine
- Sınırsız retry
- Sınırsız liste endpoint'i
- Raw GPS loglama
- Secret commit etme
- Domain modelinde JSON veya ORM tag'i
- Generic utils paketi
- Gereksiz microservice
- Gereksiz interface
- Normal hata akışında panic

29. Nihai Proje Standardı

Pet Territory Wars MVP şu standartlarla geliştirilecektir:

- Modüler monolith
- Tek stateless game engine
- Küçük ve açık paketler
- Versiyonlu RuleSet
- PostgreSQL source of truth
- Kısa transaction
- Optimistic locking
- Idempotent Walk işlemleri
- Asenkron worker akışı
- Structured logging
- Metrics ve tracing
- Test-first engine geliştirme
- Sade Git workflow
- Güvenli secret yönetimi
- Ölçüme dayalı performans optimizasyonu

Temel karar:

Mimari yeterince esnek olacak, ancak gelecekte oluşabilecek her ihtimal için baştan karmaşılaştırılmayacaktır.

Yeni abstraction, servis veya teknoloji yalnızca ölçülmüş ve gerçek bir ihtiyaç ortaya çıktığında eklenecektir.